SETUP Inicial

```python
!pip install -r requirements.txt)
```

[1] ✓ 0.8s Python

```
[notice] A new release of pip is available: 25.1.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt)'
```

BRONZE ZONE

Na camada bronze, foi realizada a conexão com as bases dentro do kagglehub, realizado o download para o .cache da maquina, posteriormente movido para dentro do diretorio do repositorio.

```python
import kagglehub
import os
import shutil
```

[2] ✓ 0.3s Python

```
d:\www\lab01_new\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_ins
  from .autonotebook import tqdm as notebook_tqdm
```

```python
if not os.path.exists('data/bronze_student_mental_health_burnout.csv'):

    path = kagglehub.dataset_download("sharmajicoder/student-mental-health-and-burnout")
    print("Path to dataset files:", path)

    files = os.listdir(path)
    csv_file = [f for f in files if f.endswith('.csv')][0]

    source = os.path.join(path, csv_file)

    dest = os.path.join('data', 'bronze_student_mental_health_burnout.csv')
    shutil.copy(source, dest)

    print("Dataset salvo em:", dest)
```

[3] ✓ 0.0s Python

SILVER ZONE

Na camada silver, foi realizado a leitura dos dados da camada bronze. Posteriormente, foi tratado estes dados e salvos em um novo arquivo em formato .parquet

Resumo dos Dados

Contagem de registros nulos por coluna

Tipo de dado por coluna

	dtype
age	int64
gender	str
academic_year	int64
study_hours_per_day	float64
exam_pressure	float64
academic_performance	float64
stress_level	float64
anxiety_score	float64
depression_score	float64
sleep_hours	float64
physical_activity	float64
social_support	float64
screen_time	float64
internet_usage	float64
financial_stress	float64
family_expectation	float64
burnout_score	float64
mental_health_index	float64
risk_level	str
dropout_risk	float64

Contagem de nulos

	null_count
age	0
gender	0
academic_year	0
study_hours_per_day	0
exam_pressure	0
academic_performance	0
stress_level	0
anxiety_score	0
depression_score	0
sleep_hours	0
physical_activity	0
social_support	0
screen_time	0
internet_usage	0
financial_stress	0
family_expectation	0
burnout_score	0
mental_health_index	0
risk_level	0
dropout_risk	0

Média e desvio padrão das colunas arredondadas

Variável	Média	Desvio Padrão
study_hours_per_day	5.00	1.99
exam_pressure	6.00	1.55
academic_performance	71.00	5.66
stress_level	4.25	1.68
anxiety_score	2.99	1.51
depression_score	1.27	1.22
sleep_hours	6.50	1.47
physical_activity	3.01	1.46
social_support	5.00	1.98
screen_time	5.02	1.96
internet_usage	5.04	2.16
financial_stress	5.00	1.98
family_expectation	5.98	1.96
burnout_score	1.78	1.66
mental_health_index	7.02	1.31
dropout_risk	1.32	1.34

```python
import re
import pandas as pd
```

[4] ✓ 1.8s Python

```python
path = 'data/bronze_student_mental_health_burnout.csv'
df = pd.read_csv(path)

#Convertendo para snake_case
def to_snake_case(name):
    name = re.sub('(.)([A-Z][a-z]+)', r'\1_\2', name)
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', name).lower()

df.columns = [to_snake_case(col) for col in df.columns]
```

[5] ✓ 1.5s Python

```python
#Convertendo para snake_case
def to_snake_case(name):
    name = re.sub('(.)([A-Z][a-z]+)', r'\1_\2', name)
    return re.sub('([a-z0-9])([A-Z])', r'\1_\2', name).lower()

df.columns = [to_snake_case(col) for col in df.columns]
```

[6] ✓ 0.0s Python

```python
#Arredondando campos

columns_to_round = ['study_hours_per_day', 'exam_pressure', 'academic_performance', 'stress_level', 'anxiety_score', 'depression_score', 'sleep_hours', 'physical_activity', 'social_su

df[columns_to_round] = df[columns_to_round].round(2)
```

[7] ✓ 0.0s Python

```python
# Removendo duplicadas
df.drop_duplicates(inplace=True)
```

[8] ✓ 0.3s Python

```python
df.to_parquet('data/silver_student_mental_health_burnout.parquet', index=False)
```

[9] ✓ 0.6s Python

Gold

Criação das tabelas fatos e dimensão. Seguido de diversas analises e geração de valor baseado nas tabelas gold.

```python
path = 'data/silver_student_mental_health_burnout.parquet'

import pandas as pd

gold_df = pd.read_parquet(path)
```

[10] ✓ 0.3s Python

```python
star_df = gold_df.copy().reset_index(drop=True)
star_df['student_id'] = star_df.index + 1

# Dimensões
# 1. Dimensão de gênero
dim_gender = star_df[['gender']].drop_duplicates().reset_index(drop=True)

dim_gender['gender_id'] = dim_gender.index + 1

# 2. Dimensão de risco
dim_risk_level= star_df[['risk_level']].drop_duplicates().reset_index(drop=True)

dim_risk_level['risk_level_id'] = dim_risk_level.index + 1

# 3. Dimensão de idade (criando faixas etárias)
bins = [0, 15, 20, 25, 30, 100]
labels = ['<15', '15-19', '20-24', '25-29', '30+']
star_df['age_group'] = pd.cut(star_df['age'], bins=bins, labels=labels, right=False)

dim_age = star_df[['age_group']].drop_duplicates().reset_index(drop=True)
dim_age['age_id'] = dim_age.index + 1

# Fato
fact = (
    star_df
    .merge(dim_gender, on='gender', how='left')
    .merge(dim_risk_level, on='risk_level', how='left')
    .merge(dim_age, on='age_group', how='left')
    .drop(columns=['gender', 'risk_level', 'age_group'])
)
```

[11] ✓ 0.2s Python

Salvando em parquet

```python
# Salvando a fato
fact.to_parquet('data/gold_fact_student.parquet', index=False)

# Salvando as dimensões
dim_age.to_parquet('data/gold_dim_age.parquet', index=False)
dim_gender.to_parquet('data/gold_dim_gender.parquet', index=False)
dim_risk_level.to_parquet('data/gold_dim_risk_level.parquet', index=False)
```

[12] ✓ 0.5s Python

Salvando no banco

```python
import sqlalchemy
import pandas as pd
```

[13] ✓ 0.0s Python

```python
#Com Sqlalchemy, podemos conectar a diversos bancos de dados.
# Aqui esta as opções para testes em postgresql, mas tambem em sqlite para validação local.

# engine = sqlalchemy.create_engine('postgresql://username:password@localhost:5432/student_mental_health')
engine = sqlalchemy.create_engine('sqlite:///student_mental_health.db')


fact.to_sql('fact_student', engine, if_exists='replace', index=False)
dim_age.to_sql('dim_age', engine, if_exists='replace', index=False)
dim_gender.to_sql('dim_gender', engine, if_exists='replace', index=False)
dim_risk_level.to_sql('dim_risk_level', engine, if_exists='replace', index=False)
```

[14] ✓ 15.4s Python

```python
query = "SELECT * FROM fact_student LIMIT 10;"
result = pd.read_sql_query(query, engine)
print(result)
```

[15] ✓ 0.0s Python

```
   age  academic_year  study_hours_per_day  exam_pressure  \
0   23              2                 5.60           6.49
1   20              3                 5.60           5.63
2   29              2                 2.58           6.02
3   27              4                 4.61           6.68
4   24              4                 2.19           4.01
5   29              3                 7.70           8.46
6   21              3                 7.01           7.29
7   23              2                 6.95           7.50
8   26              4                 6.54           6.91
9   19              3                 5.29           6.62

   academic_performance  stress_level  anxiety_score  depression_score  \
0                 68.41          4.12           2.28              1.99
1                 67.68          0.35           0.00              0.00
2                 58.37          3.48           2.43              0.85
3                 68.93          6.78           4.51              4.29
4                 69.14          1.85           1.10              0.00
5                 68.33          5.46           1.72              2.31
6                 75.18          2.99           1.45              0.00
7                 80.89          5.01           2.73              0.78
8                 77.24          5.80           4.45              2.03
9                 73.81          4.31           2.44              1.13

   sleep_hours  physical_activity  ...  internet_usage  financial_stress  \
...
8            9                  1                3                 2
9           10                  2                1                 3

[10 rows x 22 columns]
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Metricas de negocio

```python
fact = pd.read_parquet('data/gold_fact_student.parquet')
dim_age = pd.read_parquet('data/gold_dim_age.parquet')
dim_gender = pd.read_parquet('data/gold_dim_gender.parquet')
dim_risk_level = pd.read_parquet('data/gold_dim_risk_level.parquet')
```

[16] ✓ 0.0s Python

Pergunta 1

Quem dorme mais, homem ou mulheres?

```python
merged_df = fact.merge(dim_gender, on='gender_id', how='left')
summary_table = merged_df.groupby('gender')['sleep_hours'].mean().reset_index()
summary_table
```

[17] ✓ 0.0s Python

	gender	sleep_hours
0	Female	6.502638
1	Male	6.500523
2	Other	6.504899

Resposta: Valores muitos parecidos, tanto homens quanto mulheres dormem 6.5 horas por dia.

Pergunta 2

Qual faixa-etária mais propensa a burnout?

```python
merged_df = fact.merge(dim_age, on='age_id', how='left')
summary_table = merged_df.groupby('age_group')['burnout_score'].mean().reset_index()
summary_table
```

✓ 0.0s Python

	age_group	burnout_score
0	15-19	1.788881
1	20-24	1.782719
2	25-29	1.782530

Resposta: Parece contra intuitivo, mas a faixa etária mais propensa a burnout é a de 15-19 anos, seguida pela faixa de 20-24 anos. Possivelmente, pode ser devido à pressão acadêmica e social que os jovens enfrentam durante esses anos críticos de desenvolvimento.

Pergunta 3

Performace academica e tempo de estudo por dia.

```python
bins = [0, 3, 6, 10]
labels = ['Low volume of study', 'Medium volume of study', 'High  volume of study']
fact['study_group'] = pd.cut(fact['study_hours_per_day'], bins=bins, labels=labels, right=False)
summary_table = fact.groupby('study_group')['academic_performance'].mean().reset_index()
summary_table
```

✓ 0.0s Python

	study_group	academic_performance
0	Low volume of study	67.654743
1	Medium volume of study	70.536882
2	High volume of study	73.429509

Resposta: Quanto mais se estuda, maior sua perfomance academica.

Pergunta 4

Os grupos com um alto indice de score de burnout tambem terão obrigatoriamente uma um index de saude mental ruim?

```python
merged_df = fact.merge(dim_risk_level, on='risk_level_id', how='left')
summary_table = merged_df.groupby('risk_level').agg(
    burnout_score_mean=('burnout_score', 'mean'),
    burnout_score_std=('burnout_score', 'std'),
    mental_health_index_mean=('mental_health_index', 'mean'),
    mental_health_index_std=('mental_health_index', 'std')
).reset_index()
summary_table
```

✓ 0.0s Python

	risk_level	burnout_score_mean	burnout_score_std	mental_health_index_mean	mental_health_index_std
0	High	6.725353	0.649377	4.164484	0.854486
1	Low	1.044048	0.978708	7.467995	1.053973
2	Medium	4.041862	0.764560	5.657879	0.891552

Resposta: Sim, estão correlacionados, embora alguns grupos com risco baixo para burnout ainda possam ter um index mental baixo.

Pergunta 5

Comparativo entre atividade fisica e saude mental

```python
cols = [
    'study_hours_per_day', 'exam_pressure', 'academic_performance', 'stress_level',
    'sleep_hours', 'physical_activity', 'social_support', 'screen_time',
    'internet_usage', 'financial_stress', 'family_expectation'
]

correlations = fact[cols + ['risk_level_id']].corr()['risk_level_id'].drop('risk_level_id')
correlations = correlations.sort_values(ascending=False)

print("Correlation com maiores indices são as mais associadas a variavel.")
print('\n')
print(correlations)
print('\n')

worst = correlations.idxmax()
best = correlations.idxmin()
print(f"\n O que mais influencia na variavel é: {worst} = {correlations[worst]:.4f}")
```

[21] ✓ 0.2s Python

```
Correlation com maiores indices são as mais associadas a variavel.


stress_level            0.551225
exam_pressure           0.317073
study_hours_per_day     0.244519
financial_stress        0.217607
family_expectation      0.157341
academic_performance    0.041579
internet_usage         -0.000511
screen_time            -0.000676
physical_activity      -0.080116
social_support         -0.169696
sleep_hours            -0.270911
Name: risk_level_id, dtype: float64



 O que mais influencia na variavel é: stress_level = 0.5512
```

Resposta: Stress é o maior causados de burnout, mas é interessante analisar o quanto a pressão academia impacta nestes dados.